```
...c.p.d.DispatchingFunctions: The order with id 394 is packed.
...c.p.d.DispatchingFunctions: The order with id 394 is labeled.
```

The output message has been sent to the `order-dispatched` exchange, but it has not been routed to any queue because no consumer has subscribed. In the final part of this chapter, we'll complete the flow by defining a supplier in Order Service to publish messages to the `order-accepted` exchange whenever an order is accepted, and a consumer to read messages from the `order-dispatched` queue whenever an order is dispatched. But first, let's add some tests to verify the integration with the Spring Cloud Stream binder.

Before moving on, stop the application process with Ctrl-C and the RabbitMQ container with `docker-compose down`.

10.4.3 Writing integration tests with a test binder

As I have stressed several times, Spring Cloud Function and Spring Cloud Stream's whole philosophy is about keeping the application's business logic infrastructure—and middleware—neutral. After defining the original `pack()` and `label()` functions, all we did was update dependencies in Gradle and modify the configuration in application.yml.

It's a good idea to have unit tests covering the business logic, independent of the framework. But it's worth adding a few integration tests to cover the application's behavior in a Spring Cloud Stream context. You should disable the integration tests you wrote earlier in the `DispatchingFunctionsIntegrationTests` class, since now you'll want to test the integration with the external messaging system.